Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers

Anonymous Author(s)

Abstract

We present Thunderbolt, an off-chain protocol that transfers Bitcoin UTXO ownership in near-second latency, requires no channel graph, no routing, and no liquidity rebalancing, and lets the recipient be offline at the time of transfer. A single UTXO is locked once on-chain under a fixed public key; ownership then passes through an unbounded sequence of holders; each transfer is a purely off-chain, asynchronous operation whose on-chain cost is zero. The chain sees exactly two transactions regardless of how many transfers occur.

The core invariant is an algebraic cancellation: at each transfer the recipient's fresh refresh value is added to the holder's share and subtracted from the semi-trusted party's share, so both shares rotate while the on-chain key stays fixed. To enforce this, the recipient publishes an invoice to a shared append-only ledger (the *Thunderbolt Ledger*): a public commitment, an encrypted copy for himself, and an encrypted copy for the semi-trusted party, together with a zero-knowledge proof that all three encode the same refresh value. The sender fetches the invoice, verifies the proof, homomorphically folds her secret into the recipient's ciphertext to produce a new ownership credential, and publishes the result with a second zero-knowledge proof. Both proofs use a single Sigma-protocol response to force the same witness across elliptic-curve and Paillier verification equations, requiring no trusted setup. The recipient decrypts at any later time; the semi-trusted party subtracts the refresh value from its share. By binding each transfer to a distinct refresh value and context identifier, multiple UTXOs can be transferred independently in parallel.

On a laptop, a complete off-chain transfer finishes in ≈ 1 s with a combined proof size of 3.8 KB. Against general-purpose SNARK backends, even a reduced-parameter 256-bit SP1 instantiation of the same proof structure already requires 54.6 s to prove, over two orders of magnitude slower than our 471 ms native proving path; a faithful Groth16 realization would additionally require heavyweight non-native large-integer circuits and a trusted setup.

CCS Concepts

• **Security and privacy** → **Cryptography**; **Distributed systems security**.

Keywords

Bitcoin, Off-Chain Transfers, Verifiable Encryption, Paillier Cryptosystem, Zero-Knowledge Proofs, Payment Channels

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '26, The Hague, The Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/11
https://doi.org/10.1145/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2026. Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers. In *Proceedings of Proceedings of the 2026 ACM Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Bitcoin [46] intentionally constrains throughput and expressiveness at the base layer: blocks are small, scripts are limited, and every state change pays the cost of global verification. This conservatism underpins Bitcoin's credibility as a settlement layer, but makes it unsuitable as a fast, low-cost medium for everyday value transfer. The dominant response has been the *payment channel*: two (or more) parties lock funds in an on-chain UTXO (unspent transaction output), exchange signed off-chain updates, and fall back to the chain only to open or close.

The Lightning Network. The Lightning Network [51] scales payment channels to a routed graph of pre-funded, bidirectionally-online channels and is today the most widely deployed Bitcoin layer-2. However, Lightning optimizes for multi-hop routing between fixed endpoints, and some scenarios call for *asynchronous* transfers (sender and receiver need not be online simultaneously), *route-free* operation (no capacity-sufficient path is needed), and *sequential ownership migration* (the same UTXO passes through an unbounded sequence of holders), as in payroll batches, cold-wallet receipts, custody chains, and intermittently-online devices.

Statechains. Somsen's statechain protocol [57] takes a different approach: instead of routing payments across channels, it transfers entire UTXOs off-chain via a co-signing entity that shares the lock key with the current owner. At each transfer, the entity must provably delete its old co-signing key and generate a new one with the next owner. However, key deletion is fundamentally unverifiable: no cryptographic mechanism can prove that a party has forgotten a secret. Mercury Layer [21] extends statechains with blinded signing but retains the single-entity trust model and the key-deletion assumption.

The role of recipient randomness. Across these off-chain transfer designs, a decisive question is whether and how the recipient introduces a fresh random value during the transfer. In Lightning, the recipient chooses a payment preimage and publishes its hash in an invoice; the sender's payment is conditional on learning the preimage, but the preimage serves only as a release condition and does not enter the lock key itself. In statechains, the recipient plays no cryptographic role beyond receiving a new co-signing key; the transfer's security rests entirely on the entity's key deletion.

Key-splitting insight. A third possibility emerges from a property of Schnorr signatures [54]: the signing key can be additively split, so that two parties holding shares x and y can jointly sign under the public key $(x + y)\,G$ without either learning the other's share. Suppose the recipient chooses a fresh random value r and the protocol arranges for r to be *added* to one share and *subtracted*

from the other. Both shares rotate, but their sum—and therefore the public key—stays fixed. The on-chain UTXO never needs to move, yet ownership has changed hands. The remaining challenge is binding r consistently across all parties without revealing it: if a zero-knowledge proof can enforce this consistency across an elliptic-curve commitment and Paillier ciphertexts, the transfer is both verifiable and private, requiring no key deletion, no routing, and no simultaneous online requirement.

Thunderbolt. This paper presents Thunderbolt, a protocol that realizes this idea. A Bitcoin UTXO is locked once under a fixed public key jointly held by an owner and a semi-trusted party. At each transfer the recipient picks a fresh refresh value r_s, publishes an *invoice* (a public commitment, two Paillier ciphertexts, and a zero-knowledge proof of their consistency) to a shared append-only ledger (the *Thunderbolt Ledger*). The sender fetches the invoice, homomorphically folds her secret into the recipient's ciphertext, and publishes a certified credential back to the ledger. The recipient decrypts at any later time; the semi-trusted party subtracts r_s from its share. The on-chain key never changes, and the transfer is *recursive*: the new holder can immediately serve as the next sender. The full protocol is given in Section 4.

Cryptographic approach. The core challenge is ensuring that the same r_s is consistently bound across three different algebraic structures, an elliptic-curve point and two Paillier ciphertexts, without revealing r_s to anyone other than its creator. The Paillier cryptosystem [48] is essential here: its additive homomorphism operates over integers rather than group elements, so the same integer witness can appear in both an elliptic-curve discrete-log relation and a Paillier ciphertext equation. We solve the cross-domain binding with two Sigma-protocol NIZK proofs that share a common technique: a single Sigma response z forces the same integer to appear in both an EC verification equation and a Paillier verification equation. The first proof (the Invoice Consistency Proof) binds r_s across all three invoice objects; the second (the Transfer Consistency Proof) binds the sender's secret to the anchor and the new credential. Both instantiate Camenisch–Shoup verifiable encryption of discrete logarithms [15]. The semi-trusted party can be instantiated as a threshold committee via Simple VESS [56], distributing trust across n members under a standard (t, n)-honesty assumption. We compare this approach against general-purpose SNARK backends in Section 6.3.

Contributions. The main contributions of this paper are:

(1) We introduce Thunderbolt, an off-chain protocol that enables asynchronous, recursive transfer of Bitcoin UTXO ownership under a fixed on-chain lock key, mediated by a shared append-only ledger with no simultaneous online requirements.
(2) We construct two compact Sigma-protocol NIZKs (the Invoice and Transfer Consistency Proofs) that bind a single integer witness across elliptic-curve and Paillier verification equations via Camenisch–Shoup verifiable encryption, and show that the semi-trusted party can be realized as a threshold committee via Simple VESS.
(3) We evaluate Thunderbolt on a laptop, achieving near-second transfer latency with 3.8 KB proof size, and show that even a reduced-parameter 256-bit SP1 instantiation is over two orders of magnitude slower to prove.

Roadmap. Section 2 recalls background primitives (notation is summarized in Appendix A). Section 3 defines the threat model. Section 4 presents the protocol, including the design rationale, the Invoice and Transfer Consistency Proofs, and the channel lifecycle. Section 5 formalizes security via a UC ideal functionality $\mathcal{F}_{\text{Thunderbolt}}$ and analyzes attack vectors. Section 6 addresses three research questions covering transfer latency and scalability (RQ1), threshold-committee performance (RQ2), and comparison with SNARK backends (RQ3). Section 7 surveys related work and Section 8 concludes.

2 Background

This section reviews the cryptographic building blocks used by Thunderbolt. The on-chain lock key and all off-chain anchors live in an *elliptic-curve group* (Section 2.1), and the lock key is additively split between a holder and a semi-trusted party via the algebraic structure of *Schnorr signatures* (Section 2.2). Each transfer rotates both shares homomorphically through the *Paillier cryptosystem* (Section 2.3), and correctness of each rotation is certified by *Sigma-protocol* NIZKs compiled via Fiat–Shamir (Section 2.4). The core proof technique that binds an elliptic-curve commitment to a Paillier ciphertext under a single witness is *verifiable encryption of discrete logarithms* (VEDL, Section 2.5). Finally, the semi-trusted party is distributed into a threshold committee via *verifiable encrypted secret sharing* (VESS, Section 2.6).

2.1 Elliptic-Curve Groups

Let $\mathbb{G}$ be an elliptic-curve group of prime order q. We fix two independent generators of $\mathbb{G}$: the standard secp256k1 base point G, used for the on-chain lock key $P = (x_s + y_s)\,G$, and a second nothing-up-my-sleeve (NUMS) generator H (a point derived by a deterministic, publicly auditable process so that no one knows its discrete log with respect to G), used for all holder anchors $X_s = x_s H$ and invoice deltas $D_s = r_s H$. Independence of G and H (i.e., the discrete log of H with respect to G is unknown) is required so that the public anchor $X_s = x_s H$ does not reveal $x_s G$, which would allow anyone to compute $y_s G = P - x_s G$ and reduce the security of the committee share to a bare discrete-log instance. A *discrete-log pair* is a pair (x, X) with $X = x H$ for some scalar $x \in \mathbb{Z}_q$. Given only X and H, no efficient algorithm can recover x; this is the *elliptic-curve discrete-logarithm problem* (ECDLP), on which the security of Bitcoin and of our proofs rests.

2.2 Schnorr Signatures and Additive Key Splitting

A Schnorr signature [54] over a message m under public key $P = x\,G$ is a pair (R, σ) where $R = k\,G$ for a random nonce k, and $\sigma = k + e\,x$ with challenge $e = \mathsf{Hash}(R, P, m)$. Verification checks $\sigma\,G \stackrel{?}{=} R + e\,P$. Bitcoin's Taproot upgrade [62] adopts Schnorr signatures as the native signature scheme.

A key property of Schnorr signatures is that the signing key can be *additively split*: two parties holding shares x and y with $P = (x + y)\,G$ can jointly produce a valid signature under P without

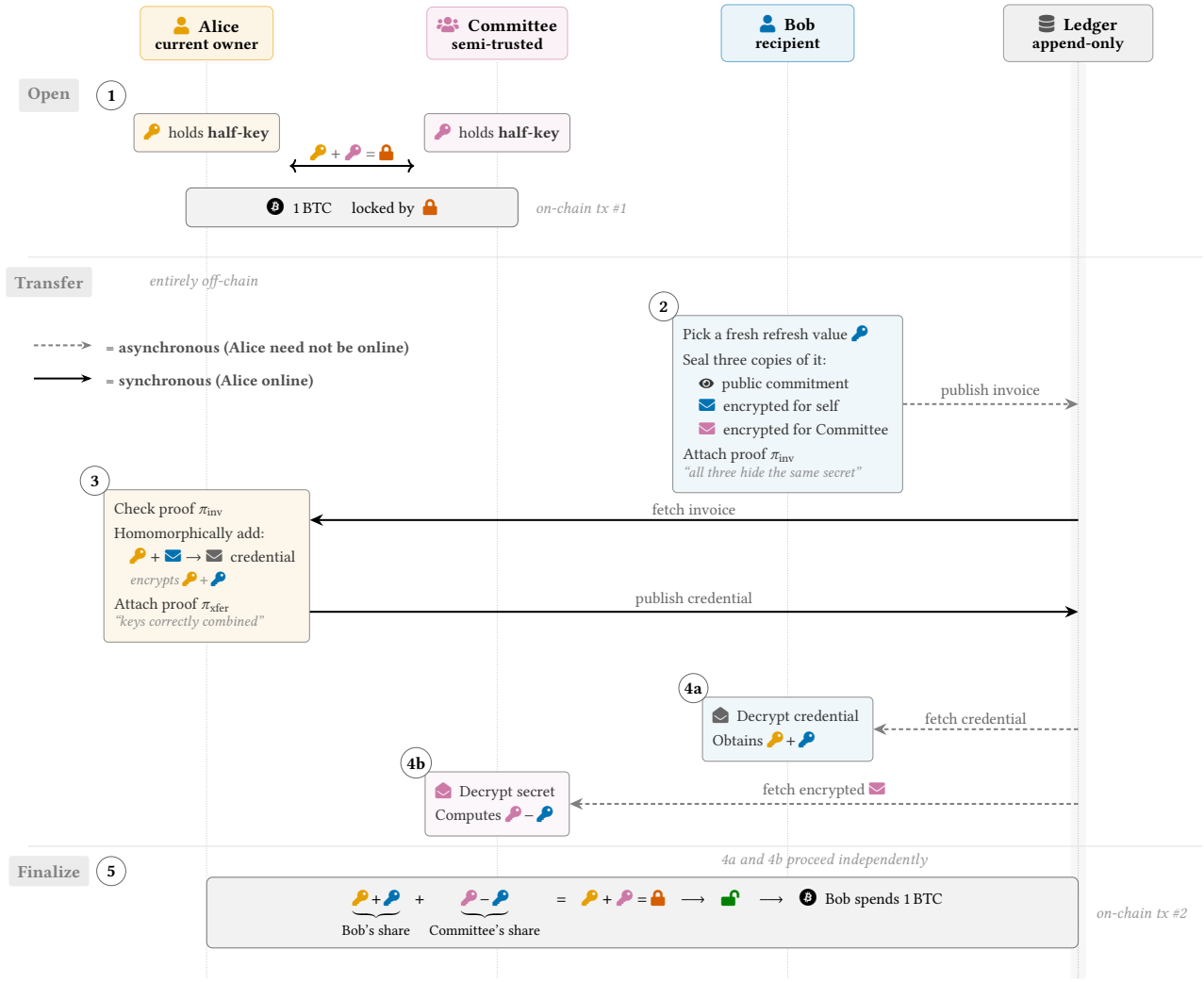


Figure 1: End-to-end overview of Thunderbolt: transferring 1 BTC from Alice to Bob via the Thunderbolt Ledger. The chain sees exactly two transactions; all transfers happen off-chain.

either party learning the other's share. Each party samples a nonce (k_1, k_2), publishes its nonce point $(R_1 = k_1 G, R_2 = k_2 G)$, computes the shared challenge $e = \text{Hash}(R_1 + R_2, P, m)$, and responds with a partial signature $(\sigma_1 = k_1 + e\,x, \sigma_2 = k_2 + e\,y)$. The combined signature $(R_1 + R_2,\ \sigma_1 + \sigma_2)$ is valid under P [41]. This additive structure is the foundation of Thunderbolt's key-sharing model: the lock key P is split between a holder and a semi-trusted party, and ownership transfer rotates both shares without changing P.

2.3 The Paillier Cryptosystem

The Paillier cryptosystem [48] is an asymmetric encryption scheme whose plaintext space is $\mathbb{Z}_N$ for an RSA modulus $N = p_1 p_2$ (two distinct primes). The public key is (N, g) with $g \in \mathbb{Z}^*_{N^2}$ of order divisible by N. Encryption of a message $m \in \mathbb{Z}_N$ with randomness $\rho \in \mathbb{Z}^*_N$ is

$$\text{Enc}_{pk}(m; \rho) = g^m \rho^N \bmod N^2.$$

Decryption uses the factorization of N. The scheme is semantically secure under the *Decisional Composite Residuosity* (DCR) assumption.

Additive homomorphism. Paillier is *additively* homomorphic: given ciphertexts $c_1 = \text{Enc}(m_1)$ and $c_2 = \text{Enc}(m_2)$, the product

$$c_1 \cdot c_2 \bmod N^2 = \text{Enc}(m_1 + m_2)$$

is a valid encryption of $m_1 + m_2 \bmod N$. We write this operation as $c_1 \oplus c_2$, and the corresponding "ciphertext subtraction" as $c_1 \ominus c_2 := c_1 \cdot c_2^{-1} \bmod N^2$. This additive homomorphism is the reason Thunderbolt can compose a fresh holder ciphertext without decrypting the invoice ciphertext.

2.4 Sigma Protocols and Fiat–Shamir

A *Sigma protocol* [22, 23] is a three-move honest-verifier zero-knowledge proof: the prover sends commitment A, receives challenge c, responds with z; the verifier checks a public equation on (A, c, z). The *Fiat–Shamir heuristic* [32] compiles it into a non-interactive zero-knowledge (NIZK) proof in the random-oracle model by setting $c = \mathsf{Hash}(\text{statement}, A)$; soundness follows from the forking lemma [50].

2.5 Verifiable Encryption of Discrete Logarithms

Camenisch and Shoup [15] give an efficient protocol for *verifiable encryption of discrete logarithms*: given a discrete-log pair $(m, M = mH)$ and a Paillier ciphertext $ct = \mathrm{Enc}_{pk}(m)$, the prover produces a Sigma-protocol proof that ct encrypts the same integer whose discrete log is M. The proof relation is

$$\mathcal{R}_{\mathrm{VE}} = \{(M, ct;\ m, \rho) : M = mH \ \wedge\ ct = g^m \rho^N \bmod N^2\}.$$

The key technique is a *shared response*: the prover commits to a single random nonce α and, after the Fiat–Shamir challenge c is derived from *all* statement components, responds with $z = \alpha + c \cdot m$. This single z must simultaneously satisfy the elliptic-curve equation ($zH \stackrel{?}{=} A_{\mathrm{EC}} + cM$) and the Paillier equation ($g^z \tau^N \stackrel{?}{=} A_{\mathrm{Pai}} \cdot ct^c$). If the integer committed in the ciphertext differed from the discrete log of M, the two equations would be mutually inconsistent for the same z. This "shared-z" construction is the technical core of our proofs.

2.6 Verifiable Encrypted Secret Sharing (VESS)

Shoup's *Simple VESS* [56] provides a mechanism for verifiably encrypting a Shamir secret sharing [55] to a committee of n shareholders, building on the line of work initiated by Feldman [31], Pedersen [49], and the distributed key-generation protocols of [34]. The sharing party (the "dealer") publishes encrypted shares $(\hat{s}_1, \ldots, \hat{s}_n)$ together with a proof that there exists a unique polynomial of degree at most $t-1$ whose evaluations are the plaintexts of these ciphertexts. Any observer can verify the consistency of the encrypted shares without decrypting them. If at least t of the n shareholders are honest, they can collectively reconstruct the shared secret.

In Thunderbolt, VESS is used when the Committee is instantiated as a threshold committee rather than a single actor. The committee-side refresh value r_s is shared via a VESS protocol so that the compensating update $y_{s+1} = y_s - r_s$ is performed collectively, without any single committee member learning y_s or r_s in the clear. This modular substitution replaces the single-committee trust assumption with a standard threshold-honesty assumption, leaving the per-step NIZK proofs unchanged.

3 Threat Model

A Thunderbolt channel involves three roles (a sequence of holders, a semi-trusted party, and the Bitcoin Chain as a passive settlement layer) communicating through a shared append-only ledger (the *Thunderbolt Ledger*), all operating under an additive key-sharing invariant. We specify the participants and communication model (Section 3.1) and state the security goals (Section 3.2).

3.1 Parties, Communication, and Trust

A Thunderbolt channel has three participant roles:

- The **Holder** (initially H_0) is the party that currently owns the UTXO off-chain. Over the lifetime of the channel, the holder role is handed from one user to the next: $\mathsf{H}_0 \to \mathsf{H}_1 \to \cdots$, where each H_s is an independent user.
- The **Semi-trusted party** C maintains the complementary share of the lock key. C never owns the UTXO and never learns the holder secret; its sole role is to preserve the fixed-key invariant via the compensating update. In practice, C is instantiated as a threshold committee $\{\mathsf{C}_1, \ldots, \mathsf{C}_n\}$ to distribute trust (Section 4.6).
- The **Bitcoin Chain** plays the role of a passive verifier: it sees exactly two transactions over the channel's lifetime (Setup and Finalize), regardless of how many holder transitions occur in between.

All off-chain communication is mediated by the **Thunderbolt Ledger**, a shared append-only bulletin board visible to all participants. Each UTXO channel is independently bound to the ledger via a distinct context identifier, so multiple UTXOs can be transferred in parallel without interference. The ledger itself is not a trusted party: it provides availability and ordering but cannot forge, suppress, or alter entries. Concretely, any authenticated broadcast medium (a smart-contract log, a federated bulletin board, or a dedicated server with append-only semantics) can instantiate the Thunderbolt Ledger.

We consider a static-corruption adversary $\mathcal{A}$ that may corrupt any subset of holders (past or future) and at most $t-1$ of the n committee members before the channel opens. The adversary is probabilistic polynomial-time (PPT) and interacts with the protocol in the random-oracle model, which Section 5 instantiates as the $(\mathcal{F}_{\mathrm{RO}}, \mathcal{F}_{\mathrm{PKI}})$-hybrid. This trust model is identical to that of standard threshold-signing deployments (FROST [40], ROAST [53]).

3.2 Security Goals

Thunderbolt aims to provide three properties:

(1) **Ownership security:** Only the current holder, the party who knows the current holder secret x_s, can authorize a transfer or participate in channel finalization.
(2) **Fixed-key integrity:** The on-chain lock key P remains unchanged throughout the channel's lifetime, regardless of how many off-chain transfers occur.
(3) **Transfer correctness:** After a valid transfer from H_s to H_{s+1}, the new holder obtains a valid holder secret x_{s+1} and the committee's share y_{s+1} is updated consistently with the fixed-key invariant.

Thunderbolt provides no liveness guarantee (fewer than t responsive members freezes the channel) and does not hide the transfer graph.

4 The Thunderbolt Protocol

This section presents the Thunderbolt protocol in full. We begin with the design rationale (Section 4.1), explaining the role of the Thunderbolt Ledger, the choice of VEDL for cross-domain binding, and the necessity of the VEDL+VESS composition. We then fix

notation and describe the complete protocol flow (Section 4.2), followed by the two core zero-knowledge proofs that enforce per-step correctness: the Invoice Consistency Proof (Section 4.3) and the Transfer Consistency Proof (Section 4.4). Their joint consequence (Section 4.5) establishes the fixed-key invariant. Finally, we show how to distribute the semi-trusted party into a threshold committee via VESS (Section 4.6).

4.1 Design Rationale

The introduction motivates the key-splitting insight and the two-phase transfer flow. This subsection explains the three design decisions that make the construction concrete: why the protocol needs a persistent ledger with per-channel binding, why VEDL is the right cross-domain proof technique, and why neither VEDL nor VESS alone suffices.

The Thunderbolt Ledger and context identifiers. Because neither the sender nor the recipient need be online at the same time, the protocol requires a persistent medium where invoices and credentials can be deposited and later retrieved. The *Thunderbolt Ledger* (Section 3.1) serves this role. Each UTXO channel is bound to the ledger via a distinct *context identifier* ctx_id that enters every Fiat–Shamir challenge hash (e.g., $c = \mathsf{Hash}(\ldots, \mathsf{ctx_id}, \ldots)$), cryptographically binding each proof to its specific channel and preventing cross-channel replay.

VEDL for cross-domain binding. The core challenge is binding the recipient's refresh value r_s consistently across three distinct algebraic structures: an elliptic-curve point and two Paillier ciphertexts. A generic Sigma protocol could prove each relation independently, but independent proofs would not prevent the prover from using different values in different domains. Verifiable encryption of discrete logarithms [15] (Section 2.5) solves this with a single shared Sigma response z that must simultaneously satisfy both an EC and a Paillier verification equation, forcing the same integer into both. We instantiate VEDL twice: the Invoice Consistency Proof extends it to a multi-receiver setting (one EC commitment and two Paillier ciphertexts bound by a single z); the Transfer Consistency Proof uses a standard single-receiver instance.

VESS for threshold trust. VEDL alone concentrates the semi-trusted party role in a single actor. VESS [56] (Section 2.6) removes this single point of failure: r_s is verifiably encrypted as a (t, n)-Shamir sharing, so that any t honest members can perform $y_{s+1} = y_s - r_s$ collectively, while no coalition of fewer than t learns anything. This substitution is modular: VESS replaces the single ciphertext ct_C with a set of verifiably encrypted shares; the Transfer Consistency Proof is unchanged, and π_{inv} narrows to the two-receiver relation (D_s, ct_B) while a separate VESS consistency proof covers the committee shares.

Necessity of the composition. Neither primitive alone suffices. VEDL without VESS forces a single trusted third party. VESS without VEDL cannot bind the shared secret to the on-chain elliptic-curve anchor. Together, VEDL enforces per-step correctness and VESS distributes trust across a threshold group.

4.2 Protocol Description

Before formalizing the consistency proofs, we describe the complete protocol flow. A Thunderbolt channel passes through three phases: **Open**, **Transfer** (zero or more), and **Finalize**—illustrated end-to-end in Figure 1. At every step s, the protocol maintains a *holder secret* x_s, a *committee secret* y_s, and a *public anchor* $X_s = x_s H$, subject to the fixed-key invariant

$$P = (x_s + y_s)\, G. \tag{1}$$

Notation. Throughout, we use the notation of Section 2.1: $\mathbb{G}$ is secp256k1 with prime order q, standard base point G (used for the lock key P), and NUMS generator H (used for all anchors and deltas). Let (N_B, g_B) and (N_C, g_C) be Paillier public keys for Bob and the Committee, respectively. All Paillier arithmetic is modulo N_B^2 or N_C^2 as appropriate. All elliptic-curve scalar equalities are taken modulo q. Let κ denote the statistical security parameter (concretely, $\kappa = 128$); we set $\tilde{q} = q^2 \cdot 2^\kappa$ as the masking range for Sigma-protocol commitment scalars, which ensures statistical zero-knowledge with distance at most $2^{-\kappa}$.

Running example. To ground the notation: suppose at step s=2 Alice holds secret x_2 with public anchor $X_2 = x_2 H$, and the committee holds y_2 such that $(x_2 + y_2)\, G = P$. Bob picks a fresh r_2, publishes three invoice objects ($D_2 = r_2 H$, $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_2)$, $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_2)$) plus proof π_{inv}. Alice folds: $T_3 = \mathrm{Enc}_{pk_B}(x_2) \oplus \mathrm{ct}_B = \mathrm{Enc}_{pk_B}(x_2 + r_2)$, updates $X_3 = X_2 + D_2$, and publishes π_{xfer}. Bob decrypts T_3 to obtain $x_3 = x_2 + r_2$; the committee computes $y_3 = y_2 - r_2$. The lock key P never changes.

Open (on-chain). The initial holder H_0 and the Committee jointly derive $P = (x_0 + y_0)\, G$ and fund a Taproot UTXO [62] locked to P; in the threshold setting, at least t committee members must acknowledge the opening. The initial anchor is $X_0 = x_0 H$ and the initial holder ciphertext is $T_0 = \mathrm{Enc}_{pk_{\mathsf{H}_0}}(x_0)$. This corresponds to step ① in Figure 1.

Transfer (off-chain). Each ownership transition is a purely off-chain operation mediated by the Thunderbolt Ledger. Denote the current holder by Alice and the next recipient by Bob. A single transfer from Alice to Bob proceeds in two phases (steps ②–④ in Figure 1):

(1) **Invoice phase (Bob).** Bob chooses a fresh refresh value $r_s \in \mathbb{Z}_q$ and publishes the following to the Thunderbolt Ledger:
- a public delta point $D_s = r_s H$,
- a ciphertext to himself, $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s;\, \rho_B)$,
- a ciphertext to the Committee, $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s;\, \rho_C)$,
- a proof π_{inv} (the Invoice Consistency Proof) that all three objects encode the same r_s.

(2) **Transfer phase (Alice).** Alice fetches the invoice from the Thunderbolt Ledger, verifies π_{inv}, forms the next holder ciphertext

$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s;\, \eta_s) \oplus \mathrm{ct}_B,$$

updates the public anchor to $X_{s+1} = X_s + D_s$, and publishes $(T_{s+1}, X_{s+1}, \pi_{\mathrm{xfer}})$ to the Thunderbolt Ledger.

After the transfer, Bob fetches T_{s+1} from the ledger, decrypts it to obtain $x_s + r_s$, and sets $x_{s+1} \equiv x_s + r_s \pmod q$. The Committee fetches and verifies both π_{inv} and π_{xfer} against the current anchor X_s, then recovers r_s from ct_C and updates $y_{s+1} = y_s - r_s$. No step requires Alice and Bob to be online simultaneously or touches the

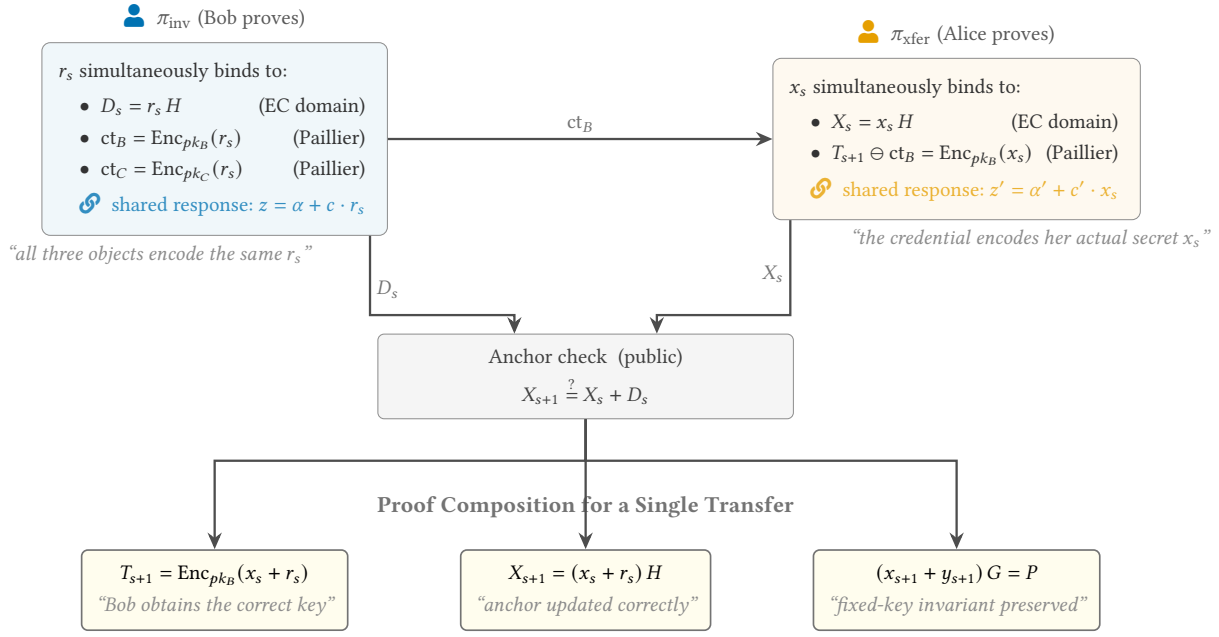


Figure 2: Proof composition for a single transfer step. π_{inv} binds r_s across three cryptographic domains via a shared response z; π_{xfer} binds x_s to the credential via a shared response z'; the anchor check links both proofs, jointly establishing the fixed-key invariant.

chain. The fixed-key invariant (1) is preserved:

$$(x_{s+1} + y_{s+1})\, G = (x_s + r_s + y_s - r_s)\, G = (x_s + y_s)\, G = P.$$

Finalize (on-chain). To close the channel, the holder first proves knowledge of the current holder secret x_s by presenting a Schnorr proof of knowledge of the discrete log of X_s with respect to H. Each committee member verifies this proof against the current anchor before agreeing to participate. The holder and the Committee then jointly sign the closing transaction under P via a standard two-party Schnorr signing protocol [41] (step ⑤ in Figure 1); in the threshold setting, at least t committee members must participate in the signing. The chain sees exactly two transactions regardless of how many transfers occurred.

By Theorem 4.5 and induction on s, the protocol preserves three invariants at every step: $(x_s + y_s)\, G = P$ (fixed key), $X_s = x_s H$ (anchor correctness), and $T_s = \text{Enc}_{pk_{\text{H}_s}}(x_s)$ (ciphertext correctness).

Security assumptions. Both proofs rely on ECDLP hardness in $\mathbb{G}$, the DCR assumption [48] for the Paillier moduli, and the random-oracle model [7] for Fiat–Shamir. Under these, both are complete, specially sound, and honest-verifier zero-knowledge (HVZK) as Sigma protocols, and NIZK after Fiat–Shamir; soundness follows from the forking lemma [50].

4.3 Invoice Consistency Proof

The invoice phase requires Bob to publish three objects encoding the same refresh value r_s: an elliptic-curve commitment D_s, a Paillier ciphertext ct_B for himself, and a Paillier ciphertext ct_C for the Committee. The Invoice Consistency Proof π_{inv} is a NIZK that forces all three to encode the same integer, preventing Bob from breaking the fixed-key invariant by committing inconsistent values.

Definition 4.1 (Invoice Consistency Relation). *The relation $\mathcal{R}_{\text{inv}}$ is defined as*

$$\mathcal{R}_{\text{inv}} = \left\{ (D_s, \text{ct}_B, \text{ct}_C;\ r_s, \rho_B, \rho_C) \ : \ \begin{array}{l} D_s = r_s H, \\ \text{ct}_B = g_B^{r_s}\, \rho_B^{N_B} \bmod N_B^2, \\ \text{ct}_C = g_C^{r_s}\, \rho_C^{N_C} \bmod N_C^2 \end{array} \right\}.$$

Sigma protocol. The prover (Bob) samples a commitment scalar $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ and commitment randomnesses $\beta_B \leftarrow \mathbb{Z}^*_{N_B}$, $\beta_C \leftarrow \mathbb{Z}^*_{N_C}$, and computes:

$$\begin{aligned} A_D &= \alpha H, \\ A_B &= g_B^{\alpha}\, \beta_B^{N_B} \bmod N_B^2, \\ A_C &= g_C^{\alpha}\, \beta_C^{N_C} \bmod N_C^2. \end{aligned}$$

Fiat–Shamir challenge. The challenge is computed as

$$c = \text{Hash}\big(\texttt{tb/invoice},\ \texttt{ctx_id},\ \texttt{inv_id},\ H,\ pk_B,\ pk_C,\ D_s,\ \text{ct}_B,\ \text{ct}_C,\ A_D,\ A_B,\ A_C\big) \bmod q.$$

Response. The prover computes a single shared response and two Paillier randomness responses:

$$z = \alpha + c \cdot r_s,$$
$$\tau_B = \beta_B \cdot \rho_B^c \bmod N_B,$$
$$\tau_C = \beta_C \cdot \rho_C^c \bmod N_C.$$

The proof is $\pi_{\text{inv}} = (A_D, A_B, A_C, z, \tau_B, \tau_C)$.

Verification. The verifier accepts iff all three of the following hold (same z in all three):

(1) **EC check:** $z\,H \stackrel{?}{=} A_D + c \cdot D_s$.
(2) **Bob–Paillier check:** $g_B^z\,\tau_B^{N_B} \stackrel{?}{=} A_B \cdot \mathrm{ct}_B^c \pmod{N_B^2}$.
(3) **Committee–Paillier check:** $g_C^z\,\tau_C^{N_C} \stackrel{?}{=} A_C \cdot \mathrm{ct}_C^c \pmod{N_C^2}$.

Theorem 4.2 (Soundness of the Invoice Consistency Proof). *The Invoice Consistency Proof is a NIZK for $\mathcal{R}_{\text{inv}}$ in the random-oracle model. In particular, the shared response z in all three verification equations forces a single integer r_s to be the discrete log of D_s and the Paillier plaintext of both ct_B and ct_C.*

The complete proof (completeness, special soundness, and honest-verifier zero-knowledge) is given in Appendix B.1.

4.4 Transfer Consistency Proof

In the transfer phase, Alice folds her current holder secret x_s into the recipient's ciphertext ct_B to produce the next holder ciphertext T_{s+1}. The Transfer Consistency Proof π_{xfer} is a NIZK that forces the folded value to match the public anchor X_s, preventing Alice from injecting an arbitrary secret that would leave Bob with an unusable credential. This proof assumes the Invoice Consistency Proof has already been verified, so D_s and ct_B are known to be consistent with a single r_s.

Definition 4.3 (Transfer Consistency Relation). *The relation $\mathcal{R}_{\text{xfer}}$ is defined as*

$$\mathcal{R}_{\text{xfer}} = \left\{ (X_s, \mathrm{ct}_B, T_{s+1};\ x_s, \eta_s) \ :\ \begin{array}{l} X_s = x_s\,H, \\ T_{s+1} \cdot \mathrm{ct}_B^{-1} = g_B^{x_s}\,\eta_s^{N_B} \bmod N_B^2 \end{array} \right\}.$$

Sigma protocol. The prover (Alice) samples a commitment scalar $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ and a Paillier commitment randomness $\beta \leftarrow \mathbb{Z}_{N_B}^*$, and computes:

$$A_X = \alpha\,H,$$
$$A_T = g_B^{\alpha}\,\beta^{N_B} \bmod N_B^2.$$

Fiat–Shamir challenge.

$$c = \mathsf{Hash}\big(\texttt{tb/xfer},\ \texttt{ctx_id},\ s{+}1,\ \texttt{inv_id},\ H,\ pk_B,\ X_s,\ \mathrm{ct}_B,\ T_{s+1},\ A_X,\ A_T\big) \bmod q.$$

Response.

$$z = \alpha + c \cdot x_s,$$
$$\tau = \beta \cdot \eta_s^c \bmod N_B.$$

The proof is $\pi_{\text{xfer}} = (A_X, A_T, z, \tau)$.

Verification. The verifier accepts iff both of the following hold (same z):

(1) **EC check:** $z\,H \stackrel{?}{=} A_X + c \cdot X_s$.
(2) **Paillier check:** $g_B^z\,\tau^{N_B} \stackrel{?}{=} A_T \cdot (T_{s+1} \ominus \mathrm{ct}_B)^c \pmod{N_B^2}$.

Additionally, the verifier checks the public anchor transition:

$$X_{s+1} \stackrel{?}{=} X_s + D_s.$$

This check is outside the zero-knowledge relation itself but is necessary for protocol correctness.

Theorem 4.4 (Soundness of the Transfer Consistency Proof). *The Transfer Consistency Proof is a NIZK for $\mathcal{R}_{\text{xfer}}$ in the random-oracle model. The shared response z forces a single integer x_s to be both the discrete log of X_s and the Paillier plaintext of $T_{s+1} \ominus \mathrm{ct}_B$.*

The complete proof follows the same structure as Theorem 4.2 and is given in Appendix B.2.

4.5 Joint Consequence: Correct Transfer Step

The two proofs above each bind a single witness to one algebraic domain; together, they guarantee that every transfer step preserves the fixed-key invariant. The following theorem makes this precise.

Theorem 4.5 (Transfer Correctness). *If both π_{inv} and π_{xfer} verify, and the anchor transition $X_{s+1} = X_s + D_s$ holds, then there exist integers x_s and r_s such that*

$$X_s = x_s\,H, \tag{2}$$
$$D_s = r_s\,H, \tag{3}$$
$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s + r_s), \tag{4}$$
$$X_{s+1} = (x_s + r_s)\,H. \tag{5}$$

Furthermore, the committee-side update $y_{s+1} = y_s - r_s$ preserves the invariant (1):

$$(x_{s+1} + y_{s+1})\,G = (x_s + y_s)\,G = P.$$

Proof. From the Invoice Consistency Proof (Theorem 4.2), extract r_s satisfying $D_s = r_s\,H$ and $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s)$. From the Transfer Consistency Proof (Theorem 4.4), extract x_s satisfying $X_s = x_s\,H$ and $T_{s+1} \ominus \mathrm{ct}_B = \mathrm{Enc}_{pk_B}(x_s)$. By Paillier's additive homomorphism, $T_{s+1} = \mathrm{Enc}_{pk_B}(x_s) \oplus \mathrm{Enc}_{pk_B}(r_s) = \mathrm{Enc}_{pk_B}(x_s + r_s)$, giving (4). The anchor transition gives $X_{s+1} = X_s + D_s = x_s\,H + r_s\,H = (x_s + r_s)\,H$, giving (5). Setting $x_{s+1} \equiv x_s + r_s \pmod q$ and $y_{s+1} \equiv y_s - r_s \pmod q$, we obtain $(x_{s+1} + y_{s+1})\,G = (x_s + y_s)\,G = P$. □

4.6 Threshold Committee via Simple VESS

The construction so far assumes a single third party C. A compromised C can collude with any holder to recover $x_s + y_s$ and steal the UTXO. We replace C with a committee of n members via Simple VESS [56]: at channel open, y_0 is distributed as a (t, n)-Shamir sharing with Feldman commitments [31]; at each transfer, Bob replaces ct_C with a VESS distribution of r_s and attaches a consistency proof. This is a *modular substitution*: the Transfer Consistency Proof remains unchanged, and π_{inv} narrows to the two-receiver relation (D_s, ct_B) while the committee-side binding is handled by the VESS consistency proof.

Proof bridging. In the threshold setting, π_{inv} covers only the two-receiver relation (D_s, ct_B). The committee-side binding comes from the VESS consistency proof, which guarantees that the encrypted shares encode a unique polynomial whose free-term commitment is F_0. The bridge is the public identity $F_0 = D_s$: π_{inv} certifies that D_s commits to r_s, and VESS certifies that the shares are consistent evaluations of the same r_s.

Committee update and finalization. Each member C_i decrypts its share $r_s^{(i)}$ and updates locally: $y_{s+1}^{(i)} = y_s^{(i)} - r_s^{(i)}$. Since both the committee polynomial and the sharing polynomial for r_s have degree $t-1$, their difference is also degree $t-1$ with free term $y_{s+1} = y_s - r_s$, preserving the Shamir structure and the fixed-key invariant. At finalization, any t members reconstruct y_s via Lagrange interpolation and jointly sign with the holder via the two-party Schnorr protocol of Section 2.2.

Security. As long as fewer than t members are corrupted, the adversary learns nothing about y_s or r_s (by information-theoretic security of Shamir sharing), and VESS verification prevents a malicious Bob from distributing inconsistent shares. The resulting trust model is the standard (t, n)-threshold honesty assumption, identical to FROST [40] and ROAST [53].

Theorem 4.6 (VESS Modularity). *The threshold-committee instantiation preserves every property of the single-third-party construction (fixed-key invariance, anchor correctness, ciphertext binding, and proof soundness) under the (t, n)-threshold honesty assumption and the security of Simple VESS [56].*

The complete proof is given in Appendix B.3.

5 Security Analysis

This section formalizes the security of Thunderbolt using the Universal Composability (UC) framework of Canetti [16]. UC analysis captures security via an *ideal functionality* $\mathcal{F}$: a trusted machine that implements the desired behavior by definition. A real-world protocol is secure if no efficient adversary can cause any observable difference between interacting with the real protocol and interacting with $\mathcal{F}$, even when the protocol is composed with arbitrary other protocols. This composability guarantee is essential for Thunderbolt, whose off-chain transfers ultimately compose with on-chain Bitcoin transactions at channel open and finalize, and may further be embedded in multi-step workflows such as atomic swaps or custody pipelines.

We define the ideal functionality $\mathcal{F}_{\text{Thunderbolt}}$ (Section 5.2), which captures the three security goals of Section 3.2 by construction. We then show that the real protocol UC-realizes $\mathcal{F}_{\text{Thunderbolt}}$ via a simulation argument (Section 5.3), and analyze concrete attack vectors (Section 5.4).

5.1 Hybrid Model

We analyze Thunderbolt in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{PKI}})$-hybrid model. $\mathcal{F}_{\text{RO}}$ is the standard random-oracle functionality [7], used by the Fiat–Shamir compilation of both NIZK proofs. $\mathcal{F}_{\text{PKI}}$ distributes Paillier public keys to all parties before the channel opens: pk_B for the recipient and $(pk_1, \ldots, pk_n)$ for the committee members (or a single pk_C in the non-threshold setting). The Thunderbolt Ledger is modeled as an authenticated append-only bulletin board absorbed by $\mathcal{F}_{\text{Thunderbolt}}$'s message-delivery interface. The adversary model follows Section 3: static corruption of any subset of holders and at most $t-1$ of the n committee members.

5.2 Ideal Functionality $\mathcal{F}_{\text{Thunderbolt}}$

$\mathcal{F}_{\text{Thunderbolt}}$ is parameterized by a prime-order group $\mathbb{G}$ and threshold (t, n). It interacts with an unbounded sequence of holder parties $\mathsf{H}_0, \mathsf{H}_1, \ldots$, committee members $\mathsf{C}_1, \ldots, \mathsf{C}_n$, the Bitcoin Chain, and the adversary $\mathcal{A}$. It maintains state $(s,\ \text{owner},\ P,\ \text{fin},\ \mathcal{P})$, initialized to $(0,\ \mathsf{H}_0,\ \bot,\ \bot,\ \emptyset)$, where $\mathcal{P}$ is the set of pending (unconsumed) invoice identifiers.

Open. On (open, P) from H_0 and (open-ack) from at least t committee members: record $P \in \mathbb{G}$, set owner $\leftarrow \mathsf{H}_0$, and send (opened, P) to all parties.

Invoice. On (invoice) from any party B: sample a fresh identifier ι, add ι to $\mathcal{P}$, record that ι was issued to B, and send (invoice-ready, ι) to owner.

Transfer. On (transfer, ι) from owner, where ι was issued to recipient B:

- If fin $\neq \bot$ or $\iota \notin \mathcal{P}$, ignore.
- Remove ι from $\mathcal{P}$ (consuming it exactly once).
- Set $s \leftarrow s + 1$ and owner $\leftarrow B$.
- Notify $\mathcal{A}$ of the state update.
- Deliver (ownership-received, s) to B when $\mathcal{A}$ schedules delivery, capturing B's asynchronous availability.

Finalize. On (finalize) from owner together with (finalize-ack) from at least t distinct committee members: set fin $\leftarrow \top$ and send (closing-tx, P, s) to the Bitcoin Chain.

$\mathcal{F}_{\text{Thunderbolt}}$ enforces by construction: only the current owner can initiate a transfer (i); the on-chain key P is fixed at open time (ii); each invoice identifier ι is consumed at most once (iii); ownership delivery is deferred to adversarial scheduling (iv); and finalization requires t honest committee members (v).

5.3 UC Realization

Theorem 5.1 (Thunderbolt UC-realizes $\mathcal{F}_{\text{Thunderbolt}}$). *The Thunderbolt protocol UC-realizes $\mathcal{F}_{\text{Thunderbolt}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{PKI}})$-hybrid model, under the ECDLP assumption in $\mathbb{G}$ and the DCR assumption for the Paillier moduli, and assuming at most $t-1$ of the n committee members are statically corrupted.*

Proof sketch. The high-level idea is: if a corrupted party can produce accepting proofs in the real protocol, the simulator can rewind the random oracle to extract the underlying secrets, showing that the adversary must have known the legitimate credentials—and therefore could not have cheated beyond what $\mathcal{F}_{\text{Thunderbolt}}$ allows.

We construct a simulator $\mathcal{S}$ and argue that no PPT environment $\mathcal{Z}$ can distinguish real from ideal execution with non-negligible probability.

Simulating proofs for honest senders. $\mathcal{S}$ holds no witness (x_s, η_s); it invokes the ZK simulators of Theorem 4.2 and Theorem 4.4 to produce transcripts $(\tilde{\pi}_{\text{inv}}, \tilde{\pi}_{\text{xfer}})$ statistically indistinguishable from real proofs, programming $\mathcal{F}_{\text{RO}}$ on fresh inputs.

Extracting from corrupted senders. If a corrupted party produces accepting proofs (π_{inv}, π_{xfer}), $\mathcal{S}$ exploits its programming authority over $\mathcal{F}_{\text{RO}}$: it records the corrupted party's commitment and programs $\mathcal{F}_{\text{RO}}$ to deliver a second distinct challenge for the same commitment on a subsequent query, obtaining two accepting transcripts with different challenges. The special-soundness extractor of Theorem 4.4 then yields x_s^* satisfying $X_s = x_s^* H$, confirming the party holds the current-owner credential; $\mathcal{S}$ then forwards (transfer, ι) to $\mathcal{F}_{\text{Thunderbolt}}$. A corrupted party unable to produce valid proofs is rejected by honest verifiers, and no transfer occurs in either world.

Simulating committee members. $\mathcal{S}$ runs the honest member algorithm with fresh random coins (indistinguishable). For fewer-than-t corrupted members, their Shamir shares are uniformly random by information-theoretic security; $\mathcal{S}$ programs them as uniform values consistent with the public verification polynomial.

Finalization. For an honest finalization, $\mathcal{S}$ reconstructs y_s from t honest shares, combines it with the extracted x_s, and produces a signature under P. An adversary attempting unilateral finalization must either compute a discrete log of P (hard by ECDLP) or recover y_s from fewer than t Shamir shares (impossible by information-theoretic secrecy).

In all cases the simulated and real transcripts are computationally indistinguishable, so $\mathcal{Z}$ cannot distinguish the two worlds with non-negligible advantage. □

5.4 Attack Vector Analysis

We discuss four concrete attack patterns and identify the mechanisms that address them.

Committee–holder collusion. If t committee members collude with the current holder H_s, they can reconstruct y_s and unilaterally spend the UTXO. This is an inherent limitation of the (t, n)-threshold trust model, identical to FROST [40] and ROAST [53]. A *past* holder $\mathsf{H}_{s'}$ is neutralized by two mechanisms: (1) finalization requires a proof of knowledge of x_s matching the *current* anchor X_s, which a past holder cannot produce; (2) fewer than t corrupted members cannot reconstruct any past $y_{s'}$ from their Shamir shares. Committee selection should follow established threshold-custody principles: diversity across organizational domains, stake or reputation with slashing, and periodic rotation via VESS resharing.

Double-spending and equivocation. A malicious holder H_s might attempt to (a) consume the same invoice twice, (b) issue valid transfer proofs to two different recipients, or (c) finalize on-chain while a transfer is in flight. Attack (a) is prevented by the *monotonic anchor*: each transfer advances X_s to $X_{s+1} = X_s + D_s$; the committee accepts a proof only against the current anchor, so a replay against the same invoice fails. Attack (b) is prevented identically: the committee processes only the first valid transfer for step s and advances its state, rejecting any subsequent proof for the same step. Attack (c) is mitigated by the cooperative nature of finalization: the holder needs t committee co-signatures and the committee can refuse to sign while a transfer is pending. All three cases correspond to $\mathcal{F}_{\text{Thunderbolt}}$ ignoring duplicate or out-of-order inputs via the single-consumption rule on invoice identifiers.

Self-transfer loops. A holder may issue an invoice to itself (e.g., $A \to B \to B \to C$). The protocol treats each step uniformly: the committee verifies proofs against the current anchor, advances its state, and updates its share. The self-transfer reduces to a holder-side key rotation and introduces no new attack surface, since exploiting any past secret still requires collusion with t committee members.

Denial-of-service by the committee. If fewer than t committee members are responsive, the channel is frozen (Section 3.2). Mitigations include redundancy ($n \gg t$), an on-chain timeout escape hatch (a timelock branch allowing the holder to unilaterally sweep after a pre-agreed period), and committee monitoring with heartbeats and VESS resharing to replace non-responsive members.

6 Evaluation

We address three research questions:

- **RQ1** (Section 6.1): How fast is a single off-chain transfer, and does per-transfer cost remain constant as the transfer count grows?
- **RQ2** (Section 6.2): How does the threshold-committee path scale with committee size and fault tolerance?
- **RQ3** (Section 6.3): How does Thunderbolt's Sigma-protocol construction compare with general-purpose SNARK backends?

Experimental setup. All benchmarks were run on a single laptop with an Apple M3 Max processor (14-core, 10 performance + 4 efficiency), 36 GB unified memory, macOS 15.2, compiled with `rustc` 1.90.0 in release mode with LTO enabled. The Thunderbolt implementation uses the `rust-paillier` crate for Paillier operations with a 3072-bit modulus ($|N| = 3072$, so all Paillier arithmetic operates over 6144-bit integers in $\mathbb{Z}_{N^2}$) and the `k256` crate for secp256k1 elliptic-curve arithmetic. The single-committee microbenchmarks report the average, minimum, and maximum over 100 iterations with a 3-iteration warmup discarded. Bob's per-member VESS share proofs are generated in parallel using `rayon`; all other steps are sequential as they would be on a single machine. The threshold-committee benchmark uses the same four committee shapes as ROAST [53] (3-of-5, 11-of-15, 34-of-50, 67-of-100) with 3 iterations, a 160 ms baseline RTT, ±20% multiplicative jitter per honest member, and a network fanout of 4 (shares are delivered in waves of 4 members, each wave separated by one baseline RTT). Non-responsive members are modeled as silent: they never reply, forcing the protocol to reach threshold from the remaining honest members alone.

6.1 RQ1: Transfer Latency and Scalability

How fast is a single Thunderbolt off-chain transfer, where is the time spent, and does per-transfer cost remain constant as the transfer count grows?

A single off-chain transfer executes nine stages: invoice setup, invoice prove, invoice verify, transfer setup, transfer prove, anchor verify, transfer verify, Bob decrypt, and committee update. Table 1 reports measured wall-clock time for each stage.

Stage	Avg (ms)	Min (ms)	Max (ms)	Share
Invoice setup	119.0	115.9	152.9	18.0%
Transfer setup	64.5	63.0	81.6	
Invoice prove	309.0	301.6	339.8	46.0%
Transfer prove	161.5	157.0	169.4	
Invoice verify	196.3	191.8	217.6	30.0%
Anchor verify	< 0.01	< 0.01	< 0.01	
Transfer verify	110.6	108.0	114.8	
Bob decrypt	30.4	29.6	33.3	6.0%
Committee update	30.4	29.5	31.9	
Total (off-chain)	**1021.8**	**998.6**	**1073.3**	**100%**

Table 1: Per-stage latency and category share for a single off-chain transfer (100 iterations, 3072-bit Paillier, secp256k1).

Cost breakdown. The total off-chain transfer completes in ≈ 1.02 **s** on average. The right side of Table 1 groups the nine stages by functional role. Invoice Prove (309 ms) and Transfer Prove (162 ms) dominate at 46% combined; both are bottlenecked by modular exponentiations over 6144-bit Paillier integers ($\mathbb{Z}_{N^2}$), while the elliptic-curve operations are negligible by comparison. This Paillier-heavy profile explains the circuit-backend overhead (Section 6.3).

Proof size. π_{inv} totals ≈ 2.3 KB (three commitments, one shared response, two randomness responses at $|N| = 3072$); π_{xfer} totals ≈ 1.5 KB; combined ≈ 3.8 KB per step.

Scalability over many transfers. Because each transfer depends only on the current state (X_s, T_s, y_s) and no party re-verifies prior transfers, per-transfer cost is $O(1)$: running 1 000 sequential transfers on a single-committee channel confirms that every transfer takes the same ≈ 1.01–1.04 s, with a constant proof size of 3.8 KB per step.

6.2 RQ2: Threshold Committee Scalability

How does the threshold-committee transfer path scale with committee size and fault tolerance?

We benchmark the full threshold-committee transfer path of Section 4.6 under the four committee configurations and network model described in the experimental setup. For each configuration, we measure the *committee communication latency*, defined as the wall-clock time from when Bob begins delivering shares until t members have responded, under three adversarial scenarios: 0, $\lceil n/6 \rceil$, and $\lceil n/3 \rceil$ non-responsive members (capped at $n-t$, the maximum tolerable), uniformly distributed across the committee. Non-responsive members model malicious or crashed participants that never reply, forcing the protocol to reach threshold from the remaining honest members alone.

Committee communication scales linearly. Committee communication latency grows linearly with n: from 0.31 s at n=5 to 2.96 s at n=100 with no adversary (Figure 3). The slope is governed by the fanout constraint: with waves of 4 members per baseline RTT, a 100-member committee requires $\lceil 100/4 \rceil = 25$ delivery waves,

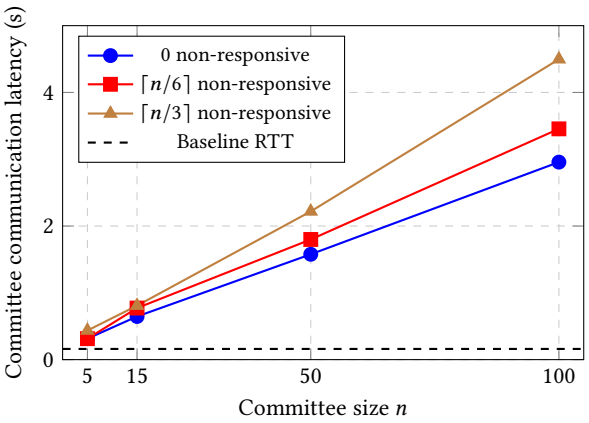


Figure 3: Committee communication latency vs. committee size under 0, $\lceil n/6 \rceil$, and $\lceil n/3 \rceil$ non-responsive members (fanout = 4, ±20% jitter, 3 iterations, 160 ms baseline RTT).

contributing ≈ 24 × 160 ms = 3.84 s of serialized network delay before the last batch begins processing.

Resilience to non-responsive members. When $\lceil n/3 \rceil$ members are non-responsive (the maximum fault tolerance), the protocol must collect t responses from the remaining honest members alone. At n=100, committee communication latency rises from 2.96 s to 4.50 s, a 52% increase (Figure 3). At n=50, the overhead is 41% (1.58 s to 2.22 s). These increases reflect the fanout cost of reaching more distant honest members: with $\lceil n/3 \rceil$ members silent and uniformly distributed, the protocol must wait for honest members in later delivery batches that would otherwise be unnecessary. Crucially, all configurations complete successfully and preserve the fixed-key invariant at every adversary level, and the $\lceil n/6 \rceil$ curve shows a moderate intermediate overhead (17% at n=100), demonstrating graceful degradation.

End-to-end transfer latency. The full threshold transfer (VESS share generation, distribution verification, transfer prove/verify, Bob decrypt, and committee communication) takes 1.72 s at n=5 and 15.98 s at n=100 with no adversary. VESS share generation dominates: Bob must compute n independent Paillier encryptions and NIZK proofs, which accounts for over 80% of the total at n=100. Committee communication, by contrast, is a bounded fraction of the end-to-end cost that degrades gracefully under adversarial conditions.

6.3 RQ3: Comparison with SNARK Backends

How does Thunderbolt's Sigma-protocol construction compare with general-purpose SNARK backends on the same relations?

Sigma protocols vs. general-purpose SNARKs. An alternative realization would encode our relations as arithmetic circuits for a SNARK backend (Groth16 [36], Plonk [33], or a zkVM such as SP1 [59] or RISC Zero [52]). The Invoice and Transfer relations are dominated by Paillier modular exponentiation in $\mathbb{Z}_{N^2}$ for a 3072-bit N, so a generic SNARK backend must handle repeated non-native arithmetic over 6144-bit values, bit decomposition and

modular reduction for ciphertext equations, and wiring the same witness consistently across both Paillier checks and the secp256k1 verification equation.

Why we do not implement a full Groth16 backend. A faithful Groth16 realization of Thunderbolt would therefore require a large custom circuit for non-native big-integer arithmetic, careful constraint engineering for modular exponentiation in $\mathbb{Z}_{N^2}$, and a per-circuit trusted setup. This is a substantial implementation burden before any performance optimization, and the resulting prover would still inherit the high cost of circuit-based proving on large-modulus arithmetic. We therefore do not implement a full Groth16 backend. This engineering gap is itself informative: Thunderbolt's statements are native algebraic relations, and forcing them into a generic SNARK circuit already erases one of the main advantages of our construction.

Reduced-parameter SP1 measurement. We instead measure the closest implemented general-purpose backend: an SP1 zkVM proof of the same invoice/transfer Paillier verification structure under a reduced 256-bit Paillier modulus. This is far smaller than the 3072-bit modulus used by Thunderbolt, but the proving cost is already high: averaged over five runs, SP1 takes 54.6 s to prove and 67 ms to verify, compared with 471 ms and 307 ms for our native Sigma proofs. That is a 116× proving slowdown even at the reduced parameter. Larger Paillier moduli were not feasible on our laptop setup because local proving exceeded the available memory budget; increasing the modulus would also further increase proving time.

Our direct Sigma-protocol construction requires no circuit: each proof consists of a constant number of native group operations. Concretely, proving both proofs takes ≈471 ms and verifying both takes ≈307 ms. Table 2 summarizes the comparison.

Backend	Prove	Verify	Slowdown
Thunderbolt (Sigma)	471 ms	307 ms	1×
SP1 zkVM (256-bit Paillier)	54.6 s	67 ms	116×

Table 2: Measured prove/verify cost per transfer. The SP1 row uses a reduced 256-bit Paillier modulus; larger moduli were not feasible within the available memory budget of our laptop setup.

Even after shrinking the Paillier modulus far below the deployed setting, the general-purpose zkVM backend remains over two orders of magnitude slower to prove. Together with the implementation complexity and trusted setup requirements of a full Groth16 circuit, this supports our design choice: Thunderbolt's algebraic relations are better handled natively by Sigma protocols than by generic SNARK backends.

7 Related Work

Payment channels and layer-2 protocols. Off-chain payment channels date to Spilman [58] and were generalized by the Lightning Network [51], duplex channels [27], and eltoo [26]. Extensions address privacy and concurrency [43], atomic multi-channel updates [29], rebalancing [39], generalized state-channel machines [45], and miner-rationality defenses [2]; Gudgeon et al. [37] and Chen et al. [19] survey the design space. Virtual channels [3, 5, 28] reduce on-chain cost by layering logical channels over existing funded ones, and channel factories [13] amortize opening transactions across many participants. Sleepy Channels [6] remove the watchtower requirement for bidirectional channels. All these designs fix the channel endpoints at open time and optimize for routing across pre-funded channels. Thunderbolt targets a complementary point: sequential ownership migration of a single UTXO with no routing, no directional capacity, and no simultaneous-online requirement, adding a (t, n)-threshold committee in place of Lightning's watchtower model.

Statechains. Somsen's statechain proposal [57] is the closest prior work: it transfers entire UTXOs off-chain via a trusted "statechain entity" that co-signs transactions. Thunderbolt differs in three respects: (1) the committee never signs off-chain transfer messages; it only maintains a secret share, while all validity is checked via public ZK proofs; (2) the fixed-key invariant shifts the key-deletion burden: whereas statechains require the statechain entity to provably destroy its old co-signing key at each transfer, Thunderbolt requires committee members to securely delete their per-step refresh shares $r_s^{(i)}$ after each update (see Section 5.4); unlike the statechain entity's unilateral deletion, committee-side deletion is enforceable by the threshold honesty assumption, since any t honest members can independently confirm that no further reconstructions of $y_{s'}$ are possible once their shares are purged; (3) threshold-committee instantiation distributes trust across n members, whereas statechains concentrate it in a single co-signer. Mercury Layer [21] extends statechains with blinded signing to improve operator privacy but retains the single-entity trust model and the key-deletion assumption.

Adaptor signatures. Adaptor signatures [4, 30] condition a presignature on the discrete log of a public point, used in PTLC-based routing and atomic swaps. Gerhart et al. [35] formalize their security for general NP relations, and Tairi et al. [60] give a composable security framework for adaptor-signature-based blockchain protocols. Thunderbolt's verifiable encryption serves a related but distinct purpose: we encrypt the discrete log itself and prove consistency via Paillier homomorphism and VEDL proofs.

Verifiable encryption and secret sharing. Camenisch and Shoup [15] give a Sigma-protocol proof that a Paillier ciphertext encrypts the discrete log of a group element, now standard in threshold cryptography, verifiable escrow [14], and optimistic fair exchange [1]. Chen et al. [18] give a UC-secure NIZK compiler from Sigma protocols, showing the composability potential of this proof family. Our Invoice and Transfer Consistency Proofs are direct instantiations of the Camenisch–Shoup technique, with the Invoice proof extending it to bind three objects (one EC point and two Paillier ciphertexts) via a single shared Sigma response. Shoup's Simple VESS [56] provides verifiable encryption of Shamir sharing [55], building on Feldman's VSS [31] and Pedersen commitments [49]. Das et al. [24] recently simplify VSS with reduced bandwidth and latency. Thunderbolt uses VESS to distribute the committee-side refresh r_s under a (t, n)-threshold group at each transfer step.

Threshold signatures for Bitcoin. FROST [40] and ROAST [53] provide efficient threshold Schnorr [54] signing; Chu et al. [20] prove FROST secure without the algebraic group model. Das et al. [25] give practical asynchronous distributed key generation, and Thyagarajan et al. [61] address threshold-service payments on Bitcoin. These protocols can finalize a Thunderbolt channel when the on-chain output is a Taproot [63] key. Lindell [41] gives a two-party ECDSA protocol, and Canetti et al. [17] a proactive threshold-ECDSA protocol with UC security. Compact multi-signatures [10] offer a complementary route to reducing on-chain key material. These threshold signing protocols are complementary to Thunderbolt: they handle the on-chain finalization (the Finalize phase), while our Sigma-protocol proofs handle the off-chain state transitions.

General-purpose zero-knowledge proofs. Groth16 [36], Plonk [33], transparent STARKs [8], and zkVM-based systems such as Succinct SP1 [59] and RISC Zero [52] can in principle prove any NP statement. While powerful, these systems pay a large constant for algebraic relations that Sigma protocols handle natively. Recursive proof composition [11] and accumulation-based aggregation [9] reduce verification costs but do not eliminate proving-time overhead for Paillier-heavy statements.

Off-chain protocols beyond Bitcoin. Ethereum's layer-2 landscape includes optimistic rollups [47], zk-rollups [44], state channels [28, 45], and commit-chains [38]. These systems leverage Ethereum's expressive smart-contract layer to enforce state transitions on-chain when needed, an option unavailable in Bitcoin's restricted scripting model. In the Bitcoin context, Ark [12] proposes virtual UTXOs managed by a service provider who periodically settles on-chain, and BitVM [42] enables general computation verification via fraud proofs. Thunderbolt occupies a different point in the design space: it targets single-UTXO ownership transfer using only standard Taproot, requiring no new opcodes, no fraud-proof infrastructure, and no expressive scripting beyond what Bitcoin already provides.

8 Conclusion

We presented Thunderbolt, an off-chain Bitcoin protocol that transfers UTXO ownership asynchronously and recursively under a fixed on-chain lock key. The construction pairs two well-studied primitives (Camenisch–Shoup verifiable encryption and Simple VESS) in a new way: two Sigma-protocol NIZKs enforce the "one add, one subtract" invariant per step, while VESS distributes the committee share under a standard (t, n)-threshold-honesty assumption. A single transfer completes in near-second latency with 3.8 KB proof size; per-transfer cost is $O(1)$ in transfer count; the threshold path scales linearly with committee size (2.96 s committee communication at n=100) and degrades gracefully under adversarial conditions, completing successfully even when $\lceil n/3 \rceil$ members are non-responsive.

Complementary to Lightning's routed payment-channel network, Thunderbolt provides asynchronous, route-free ownership transfer: a transfer requires only a one-time invoice from the recipient, and the sender can consume it at any later time without coordination. This makes Thunderbolt well-suited for payroll batches, cold-wallet receipts, custody chains, and low-power devices that wake intermittently.

Thunderbolt introduces no new cryptographic assumption; both primitives have been scrutinized for over two decades. The contribution is a new *usage scenario* for these building blocks: asynchronous, recursive, off-chain Bitcoin ownership transfer under a fixed on-chain footprint.

Future directions. Several avenues remain open. First, a concrete analysis of specific composed systems (atomic cross-chain swaps, multi-channel constructions) would characterize the composed ideal functionalities and their interaction with Thunderbolt's trust assumptions. Second, end-to-end validation with geographically distributed committee members over real network links is needed; committee-communication results (Section 6.2) suggest the bottleneck will remain Bob's VESS share distribution, which scales linearly with committee size. Third, the state-compaction mechanism (committee-signed checkpoints) requires a concrete protocol design, including the conditions under which a checkpoint safely replaces the full transfer history and how refresh-share deletion (Section 5.4) interacts with the checkpointing policy. Finally, a routed graph of Thunderbolt channels could combine Lightning's multi-hop reach with Thunderbolt's asynchronous, route-free endpoints.

References

[1] N. Asokan, Victor Shoup, and Michael Waidner. 1998. Optimistic Fair Exchange of Digital Signatures. In *Advances in Cryptology — EUROCRYPT '98 (LNCS, Vol. 1403)*. Springer, 591–606. doi:10.1007/BFb0054156

[2] Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and Subhra Mazumdar. 2024. Securing Lightning Channels against Rational Miners. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 393–407. doi:10.1145/3658644.3670373

[3] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Bitcoin-Compatible Virtual Channels. In *2021 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 901–918. doi:10.1109/SP40001.2021.00097

[4] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures. In *Advances in Cryptology — ASIACRYPT 2021 (LNCS, Vol. 13091)*. Springer, 635–666. doi:10.1007/978-3-030-92075-3_22

[5] Lukas Aumayr, Pedro Moreno-Sanchez, Aniket Kate, and Matteo Maffei. 2023. Breaking and Fixing Virtual Channels: Domino Attack and Donner. In *30th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2023.24370

[6] Lukas Aumayr, Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, Pedro Moreno-Sanchez, and Matteo Maffei. 2022. Sleepy Channels: Bi-directional Payment Channels without Watchtowers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2521–2535. doi:10.1145/3548606.3559370

[7] Mihir Bellare and Phillip Rogaway. 1993. Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS)*. ACM, 62–73. doi:10.1145/168588.168596

[8] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2019. Scalable Zero Knowledge with No Trusted Setup. In *Advances in Cryptology — CRYPTO 2019 (LNCS, Vol. 11694)*. Springer, 701–732. doi:10.1007/978-3-030-26954-8_23

[9] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Recursive Proof Composition from Accumulation Schemes. Cryptology ePrint Archive, Report 2020/499. https://eprint.iacr.org/2020/499

[10] Dan Boneh, Manu Drijvers, and Gregory Neven. 2018. Compact Multi-Signatures for Smaller Blockchains. In *Advances in Cryptology — ASIACRYPT 2018 (LNCS, Vol. 11273)*. Springer, 435–464. doi:10.1007/978-3-030-03329-3_15

[11] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Halo: Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021. https://eprint.iacr.org/2019/1021

[12] Burak. 2023. Ark: A Second-Layer Protocol for Bitcoin. White paper. https://arkdev.info/

[13] Conrad Burchert, Christian Decker, and Roger Wattenhofer. 2018. Scalable Funding of Bitcoin Micropayment Channel Networks. In *Royal Society Open Science*, Vol. 5. doi:10.1098/rsos.180089

[14] Jan Camenisch and Anna Lysyanskaya. 2001. Efficient Non-transferable Anonymous Multi-show Credential System with Optional Anonymity Revocation. In *Advances in Cryptology — EUROCRYPT 2001 (LNCS, Vol. 2045)*. Springer, 93–118. doi:10.1007/3-540-44987-6_7

[15] Jan Camenisch and Victor Shoup. 2003. Practical Verifiable Encryption and Decryption of Discrete Logarithms. In *Advances in Cryptology — CRYPTO 2003 (LNCS, Vol. 2729)*. Springer, 126–144. doi:10.1007/978-3-540-45146-4_8

[16] Ran Canetti. 2001. Universally Composable Security: A New Paradigm for Cryptographic Protocols. In *42nd IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE, 136–145. doi:10.1109/SFCS.2001.959888

[17] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1769–1787. doi:10.1145/3372297.3423367

[18] Megan Chen, Pousali Dey, Chaya Ganesh, Pratyay Mukherjee, Pratik Sarkar, and Swagata Sasmal. 2025. Universally Composable Non-interactive Zero-Knowledge from Sigma Protocols via a New Straight-Line Compiler. In *Public-Key Cryptography — PKC 2025*. Springer, 381–417. doi:10.1007/978-3-031-91820-9_13

[19] Xiuyuan Chen, Junzuo Lai, Chao Lin, Xinyi Huang, and Debiao He. 2025. Cryptographic Primitives in Script-based and Scriptless Payment Channel Networks: A Survey. *Comput. Surveys* (2025), 1–34. doi:10.1145/3725846

[20] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. 2023. Practical Schnorr Threshold Signatures Without the Algebraic Group Model. In *Advances in Cryptology — CRYPTO 2023 (LNCS, Vol. 14081)*. Springer, 743–773. doi:10.1007/978-3-031-38557-5_24

[21] CommerceBlock. 2023. Mercury Layer: Statechain Implementation with Blinded Signing. Project documentation. https://mercurylayer.com/

[22] Ronald Cramer. 1997. *Modular Design of Secure yet Practical Cryptographic Protocols*. Ph.D. Thesis. University of Amsterdam.

[23] Ivan Damgård. 2010. On Σ-Protocols. Lecture notes, Aarhus University. https://www.cs.au.dk/~ivan/Sigma.pdf

[24] Sourav Das, Zhuolun Xiang, Alin Tomescu, Alexander Spiegelman, Benny Pinkas, and Ling Ren. 2025. Verifiable Secret Sharing Simplified. In *2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 633–651. doi:10.1109/SP61157.2025.00046

[25] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. 2022. Practical Asynchronous Distributed Key Generation. In *2022 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2518–2534. doi:10.1109/SP46214.2022.9833584

[26] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. 2018. eltoo: A Simple Layer2 Protocol for Bitcoin. White paper. https://blockstream.com/eltoo.pdf

[27] Christian Decker and Roger Wattenhofer. 2015. A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels. In *Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS, Vol. 9212)*. Springer, 3–18. doi:10.1007/978-3-319-21741-3_1

[28] Stefan Dziembowski, Sebastian Faust, and Kristina Hostáková. 2018. General State Channel Networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 949–966. doi:10.1145/3243734.3243856

[29] Christoph Egger, Pedro Moreno-Sanchez, and Matteo Maffei. 2019. Atomic Multi-Channel Updates with Constant Collateral in Bitcoin-Compatible Payment-Channel Networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 801–815. doi:10.1145/3319535.3345666

[30] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. 2021. Two-Party Adaptor Signatures from Identification Schemes. In *Public-Key Cryptography — PKC 2021 (LNCS, Vol. 12710)*. Springer, 451–480. doi:10.1007/978-3-030-75245-3_17

[31] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS)*. 427–438. doi:10.1109/SFCS.1987.4

[32] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO '86 (LNCS, Vol. 263)*. Springer, 186–194. doi:10.1007/3-540-47721-7_12

[33] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PlonK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. https://eprint.iacr.org/2019/953

[34] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. 2007. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *Journal of Cryptology* 20, 1 (2007), 51–83. doi:10.1007/s00145-006-0347-3

[35] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. 2024. Foundations of Adaptor Signatures. In *Advances in Cryptology — EUROCRYPT 2024 (LNCS, Vol. 14654)*. Springer, 161–189. doi:10.1007/978-3-031-58723-8_6

[36] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology — EUROCRYPT 2016 (LNCS, Vol. 9666)*. Springer, 305–326. doi:10.1007/978-3-662-49896-5_11

[37] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. 2020. SoK: Layer-Two Blockchain Protocols. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 12059)*. Springer, 201–226. doi:10.1007/978-3-030-51280-4_12

[38] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: Scalable, Private Smart Contracts. In *27th USENIX Security Symposium*. USENIX, 1353–1370.

[39] Rami Khalil and Arthur Gervais. 2017. Revive: Rebalancing Off-Blockchain Payment Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 439–453. doi:10.1145/3133956.3134033

[40] Chelsea Komlo and Ian Goldberg. 2021. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In *Selected Areas in Cryptography (SAC) (LNCS, Vol. 12804)*. Springer, 34–65. doi:10.1007/978-3-030-81652-0_2

[41] Yehuda Lindell. 2017. Fast Secure Two-Party ECDSA Signing. In *Advances in Cryptology — CRYPTO 2017 (LNCS, Vol. 10402)*. Springer, 613–644. doi:10.1007/978-3-319-63715-0_21

[42] Robin Linus. 2023. BitVM: Compute Anything on Bitcoin. White paper. https://bitvm.org/bitvm.pdf

[43] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 455–471. doi:10.1145/3133956.3134096

[44] Matter Labs. 2023. zkSync: Scaling Freedom for All. Project documentation. https://zksync.io/

[45] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks that Go Faster than Lightning. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 11598)*. Springer, 508–526. doi:10.1007/978-3-030-32101-7_30

[46] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. White paper. https://bitcoin.org/bitcoin.pdf

[47] Offchain Labs. 2021. Arbitrum: Scalable, Private Smart Contracts. Project documentation. https://developer.arbitrum.io/

[48] Pascal Paillier. 1999. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. 1592 (1999), 223–238. doi:10.1007/3-540-48910-X_16

[49] Torben Pryds Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology — CRYPTO '91 (LNCS, Vol. 576)*. Springer, 129–140. doi:10.1007/3-540-46766-1_9

[50] David Pointcheval and Jacques Stern. 1996. Security Proofs for Signature Schemes. In *Advances in Cryptology — EUROCRYPT '96 (LNCS, Vol. 1070)*. Springer, 387–398. doi:10.1007/3-540-68339-9_33

[51] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. White paper. https://lightning.network/lightning-network-paper.pdf

[52] RISC Zero. 2024. RISC Zero: General-Purpose Verifiable Computing. Project documentation. https://www.risczero.com/

[53] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. 2022. ROAST: Robust Asynchronous Schnorr Threshold Signatures. Cryptology ePrint Archive, Report 2022/550. https://eprint.iacr.org/2022/550

[54] Claus-Peter Schnorr. 1991. Efficient Signature Generation by Smart Cards. *Journal of Cryptology* 4, 3 (1991), 161–174. doi:10.1007/BF00196725

[55] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. doi:10.1145/359168.359176

[56] Victor Shoup. 2025. Simple VESS. Cryptology ePrint Archive, Report 2025/1175. https://eprint.iacr.org/2025/1175

[57] Ruben Somsen. 2020. Statechains: Non-custodial Off-chain Bitcoin Transfer. White paper. https://github.com/RubenSomsen/rubensomsen.github.io/blob/master/img/statechains.pdf

[58] Jeremy Spilman. 2013. Anti DoS for tx replacement. Bitcoin-development mailing list. https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2013-April/002433.html

[59] Succinct Labs. 2024. SP1: A zkVM for Developers. Project documentation. https://docs.succinct.xyz/

[60] Erkan Tairi, Pedro Moreno-Sanchez, and Clara Schneidewind. 2023. LedgerLocks: A Security Framework for Blockchain Protocols Based on Adaptor Signatures. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 859–873. doi:10.1145/3576915.3623149

[61] Sri AravindaKrishnan Thyagarajan, Easwar Vivek Mangipudi, Lucjan Hanzlik, Aniket Kate, and Pratyay Mukherjee. 2025. Vitārit: Paying for Threshold Services on Bitcoin and Friends. In *2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2018–2036. doi:10.1109/SP61157.2025.00170

[62] Pieter Wuille, Jonas Nick, and Tim Ruffing. 2020. BIP 340: Schnorr Signatures for secp256k1. Bitcoin Improvement Proposals. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[63] Pieter Wuille, Jonas Nick, and Anthony Towns. 2020. BIP 341: Taproot: SegWit Version 1 Spending Rules. Bitcoin Improvement Proposals. https://github.com/

bitcoin/bips/blob/master/bip-0341.mediawiki

Ethical Considerations

This work presents a cryptographic protocol for off-chain Bitcoin transfers. The protocol employs only standard primitives (Paillier encryption, Schnorr-based Sigma protocols, Shamir secret sharing) that have been publicly studied for over two decades. No user data was collected or processed; all experiments use synthetic cryptographic inputs on a locally controlled machine. The protocol does not introduce new mechanisms for bypassing existing blockchain compliance or anti-money-laundering frameworks; it operates within the same trust model as existing custody and multi-signature deployments. We see no direct ethical hazards beyond those already present in any privacy-enhancing cryptocurrency technology.

Open Science Policy

A Notation Summary

Symbol	Meaning
$\mathbb{G}$	Elliptic-curve group (secp256k1), prime order q
G	Standard base point of $\mathbb{G}$ (lock key domain)
H	NUMS generator of $\mathbb{G}$ (anchor / delta domain)
$P = (x_s + y_s)\,G$	Fixed on-chain lock key
x_s	Holder secret at step s
y_s	Committee secret at step s
r_s	Refresh value chosen by the recipient at step s
$X_s = x_s\,H$	Public anchor at step s
$D_s = r_s\,H$	Public delta (anchor increment) at step s
(N, g)	Paillier public key; $N = p_1 p_2$
$\mathrm{Enc}_{pk}(m;\rho)$	Paillier encryption of m with randomness ρ
$\oplus, \ominus$	Homomorphic ciphertext addition / subtraction
ct_B, ct_C	Invoice ciphertexts for Bob and Committee
T_s	Holder ciphertext at step s (encrypts x_s)
π_{inv}	Invoice Consistency Proof
π_{xfer}	Transfer Consistency Proof
κ	Statistical security parameter ($\kappa = 128$)
$\tilde{q} = q^2 \cdot 2^{\kappa}$	Masking range for Sigma commitment scalars
(t, n)	Threshold parameters for the committee
ctx_id	Per-channel context identifier

Table 3: Summary of notation used throughout the paper.

B Proof Details

B.1 Proof of Theorem 4.2 (Invoice Consistency)

Proof. We establish the three standard properties.

Completeness. If the prover is honest, then $z = \alpha + c\,r_s$, $\tau_B = \beta_B \cdot \rho_B^c$, and $\tau_C = \beta_C \cdot \rho_C^c$. For the EC check: $z\,H = (\alpha + c\,r_s)\,H = \alpha\,H + c\,r_s\,H = A_D + c\,D_s$. For the Bob–Paillier check: $g_B^z\,\tau_B^{N_B} = g_B^{\alpha + c\,r_s}\,(\beta_B \cdot \rho_B^c)^{N_B} = (g_B^{\alpha}\,\beta_B^{N_B}) \cdot (g_B^{r_s}\,\rho_B^{N_B})^c = A_B \cdot \mathrm{ct}_B^c$. The Committee–Paillier check is analogous.

Special soundness. Given two accepting transcripts $(A_D, A_B, A_C,\ c, z, \tau_B, \tau_C)$ and $(A_D, A_B, A_C,\ c', z', \tau_B', \tau_C')$ with $c \neq c'$, define $\Delta c = c - c'$ and $\Delta z = z - z'$ (computed as integers; since $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ with $\tilde{q} = q^2 \cdot 2^{\kappa}$ and $c, c' \in \mathbb{Z}_q$, the response $z = \alpha + c\,r_s$ lies in $\mathbb{Z}_{\tilde{q}}$ without wrapping around). Because $\Delta z / \Delta c = r_s$ holds as an exact integer (the masking range ensures no modular cancellation), we extract: $r_s^* := \Delta z / \Delta c \in \mathbb{Z}$ (reducing modulo q gives the EC scalar satisfying $r_s^*\,H = D_s$). From the Bob–Paillier equation: $g_B^{\Delta z} \cdot (\tau_B / \tau_B')^{N_B} = \mathrm{ct}_B^{\Delta c}$, from which we extract $\rho_B^* = (\tau_B / \tau_B')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $\mathrm{ct}_B = g_B^{r_s^*} \cdot (\rho_B^*)^{N_B}$ (the same integer r_s^* appears in the exponent, since the shared Δz drives both equations). The Committee–Paillier extraction is identical, yielding ρ_C^*. Because a single Δz drives all three extractions, the extracted witness r_s^* is the same integer in all three relations, establishing $(D_s, \mathrm{ct}_B, \mathrm{ct}_C;\ r_s^*, \rho_B^*, \rho_C^*) \in \mathcal{R}_{\mathrm{inv}}$.

Honest-verifier zero-knowledge. The simulator $\mathcal{S}$, given the statement $(D_s, \mathrm{ct}_B, \mathrm{ct}_C)$, samples $z \leftarrow \mathbb{Z}_{\tilde{q}}$, $\tau_B \leftarrow \mathbb{Z}_{N_B}^*$, $\tau_C \leftarrow \mathbb{Z}_{N_C}^*$, programs the random oracle to return c, and sets: $A_D = z\,H - c\,D_s$, $A_B = g_B^z\,\tau_B^{N_B} \cdot \mathrm{ct}_B^{-c} \bmod N_B^2$, $A_C = g_C^z\,\tau_C^{N_C} \cdot \mathrm{ct}_C^{-c} \bmod N_C^2$. The simulated transcript $(A_D, A_B, A_C, c, z, \tau_B, \tau_C)$ is statistically indistinguishable from an honest transcript (statistical distance at most $2^{-\kappa}$), since the masking range $\tilde{q}$ absorbs the shift $c \cdot r_s$.

Soundness in the non-interactive (Fiat–Shamir compiled) setting follows from the forking lemma [50] applied in the random-oracle model. □

B.2 Proof of Theorem 4.4 (Transfer Consistency)

Proof. Let $U = T_{s+1} \ominus \mathrm{ct}_B = T_{s+1} \cdot \mathrm{ct}_B^{-1} \bmod N_B^2$ for brevity.

Completeness. With $z = \alpha + c\,x_s$ and $\tau = \beta \cdot \eta_s^c$: $z\,H = \alpha\,H + c\,x_s\,H = A_X + c\,X_s$, and $g_B^z\,\tau^{N_B} = g_B^{\alpha}\,\beta^{N_B} \cdot (g_B^{x_s}\,\eta_s^{N_B})^c = A_T \cdot U^c$.

Special soundness. Given two accepting transcripts with challenges $c \neq c'$, define $\Delta c = c - c'$, $\Delta z = z - z'$ (as integers; the masking range $\tilde{q}$ ensures no wrap-around). We extract the witness as the exact integer $x_s^* := \Delta z / \Delta c \in \mathbb{Z}$ (reducing modulo q gives the EC scalar satisfying $X_s = x_s^*\,H$). From the Paillier equation: $\eta_s^* = (\tau / \tau')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $U = g_B^{x_s^*}\,(\eta_s^*)^{N_B} \bmod N_B^2$ (the same integer x_s^* appears in the Paillier exponent because it is derived from the shared Δz). Together, $(X_s, U;\ x_s^*, \eta_s^*) \in \mathcal{R}_{\mathrm{xfer}}$.

Honest-verifier zero-knowledge. The simulator samples $z \leftarrow \mathbb{Z}_{\tilde{q}}$, $\tau \leftarrow \mathbb{Z}_{N_B}^*$, programs c via the random oracle, and sets $A_X = z\,H - c\,X_s$ and $A_T = g_B^z\,\tau^{N_B} \cdot U^{-c} \bmod N_B^2$. The resulting transcript is statistically indistinguishable from an honest one (statistical distance at most $2^{-\kappa}$).

As with the Invoice proof, non-interactive soundness follows from the forking lemma [50]. □

B.3 Proof of Theorem 4.6 (VESS Modularity)

Proof. The Transfer Consistency Proof π_{xfer} depends only on D_s, ct_B, X_s, and T_{s+1}; none of these are affected by the VESS substitution, so π_{xfer} remains sound and zero-knowledge as before. In the threshold setting, π_{inv} is run on the two-receiver relation (D_s, ct_B), which is a strict sub-relation of $\mathcal{R}_{\mathrm{inv}}$; its soundness and ZK properties follow by the same shared-z argument of Theorem 4.2. The committee-side binding is provided instead by the VESS consistency proof: VESS verification guarantees that the encrypted shares are evaluations of a unique degree-$(t-1)$ polynomial f with $f(0) = r_s$, where $r_s\,H = F_0 = D_s$ is the public free-term commitment. A malicious Bob who distributes inconsistent shares is

detected by any observer, and a Bob who sets $F_0 \neq D_s$ is rejected by π_{inv}. Correctness of the committee-side update follows from the Shamir-preservation argument above. Secrecy of y_s and r_s holds information-theoretically as long as fewer than t members are corrupted. □